IndiCare

Mobile IoT monitoring

Technical report and evaluation

Synthetic health classification and
consent based caregiver communication

Team members	Nathan Ratichon	2362789
	Guannan Ge	2364229
	Jose Manuel Cortes Ceron	2686162
	Cheng Zhang	2361292
Tutor	Matilda Isaac	

Report date	27 October 2026
Repository	`https://github.com/deepdevjose/Older-Monitoring-Wearable`

Educational use only
This prototype must not be used for diagnosis, treatment or clinical decision making. Evaluation uses synthetic fixtures. Results distinguish server experiments, Android interface tests and hardware observations.

Contents

1 Purpose and assessment scope **1**

2 Requirements and system architecture **1**

3 Hardware and data boundaries **3**

4 Mobile behaviour and programming structure **4**

5 REST integration storage and alert lifecycle **6**

6 Synthetic classification and model evaluation **7**

7 Security configuration and development evidence **8**

8 Testing measured outcomes and improvement **10**

9 Conclusion **11**

References **12**

A API contract and synthetic example **13**

B Reproduction installation and scenario evidence **14**

C Historical development images **18**

D Project team and development history **19**

E Demonstration protocol and evaluation boundaries **20**

1 Purpose and assessment scope

IndiCare examines how an Android gateway, a local REST service and introductory machine learning support an educational monitoring workflow. The application is named CareMonitor 1.0.0. It combines persistent synthetic alert episodes, consent based access, historical charts and saved synthetic classifications with confidence. The current revision connects these features to native Android screens. Evidence distinguishes current software checks from the dated Samsung onboarding campaign, isolated HTTPS experiments and hardware observations. Each result is attributed to the layer actually exercised.

The assessment covers patient and caregiver views, three class predictions, confidence, storage, alerts and measurable improvement. Mobile functionality and programming architecture carry 45% of the rubric, making the distinction between server capabilities and visible application behaviour central to the evaluation. This report maps requirements to code, presents quantitative results and analyses the implemented system's coverage. The assessment uses the current versioned API and release records rather than treating historical development captures as proof of current behaviour.[1, 2]

Classification uses synthetic educational labels. Participant measurements, clinical records and induced falls are outside this study. Historical screenshots document technical development without presenting personal physiological values. A separate SisFall experiment is excluded from the reported classification results. The evaluated CSV is identified as a repository dataset; institutional approval or equivalence to a module supplied dataset is not inferred from its presence. This preserves the distinction between reproducibility, assessment provenance and clinical validity.

2 Requirements and system architecture

The primary actors are an adult account, corresponding to the rubric's patient role, and an explicitly authorised caregiver. An adult can approve several caregivers and revoke their access independently. Requirements follow a reading through provenance, validation, transfer, persistence, presentation and alert review. Table 1 associates these behaviours with implementation locations and the evidence available for each one.

Figure 1 distinguishes two implemented mobile and server paths. The care interface retrieves readings, historical metric aggregates and rule alerts. Learning insights, reached through authenticated Settings, submits synthetic feature vectors and displays saved classifications with confidence. Its encrypted records occupy a separate table; neither synthetic model exercises nor confidence scores become device readings or care notifications. This separation supports assessment of the model while preserving the diagnostic source of wearable observations.

A local FastAPI service keeps Python validation, models and account authorisation together. Central inference avoids distributing model updates to every phone but depends on server availability. SQLite offers transactional episode creation with modest deployment complexity. Its single writer architecture suits the evaluated scale; the experiments do not establish large deployment capacity. Native Android Java provides Bluetooth and foreground service access without introducing a cloud service. This describes the technical choice without inferring administrative approval of a particular framework.

The optional ESP32 path supplies device diagnostics over Bluetooth Serial Port Profile to the Android gateway. Source labels distinguish diagnostics from synthetic assessment data. Server inference rejects device sources because synthetic body temperature is not interchangeable with the available surface temperature measurement. HTTPS protects phone to server communication; Bluetooth pairing forms a separate trust boundary. Account and resource authorisation apply independently of either transport.

Observable invariants guide evaluation. A retry preserves one record per sample, and sustained abnormal input preserves one active episode per signal. Unauthorised requests disclose no patient data, while unavailable predictions retain an explicit unknown or error state. Accessibility evidence covers enlarged registration text and control geometry on the installed phone. Timing results are reported as measured

baselines because the supplied specification defines no numeric response or delivery threshold. A measured value is consequently distinguished from a formally satisfied timing requirement.

Table 1. Requirements traced to implementation and evidence. S: source inspected; E: backend or JVM executed; D: dated production-package checks on a phone; F: current isolated preview UI fixture on a phone.

ID	Requirement and component	Verification and scope	State
R1	Older adult and caregiver views; MainActivity	Role selection, navigation and caregiver wearable guard checked on device	E/D/F
R2	Reading storage and retrieval; readings API, SQLite	Idempotency, access and ordering tests pass	E
R3	Tappable metric history; history API, MetricHistoryChartView	Mounted Day/Week/Month charts; weighted aggregation, gaps and consent tested	E/F
R4	Three classes and confidence; ModelService, LearningActivity	Class, score and saved history in UI fixture; persistence and access tested separately	E/F
R5	Stored alerts and review; alerts, private inbox	Sustained episode, recovery and independent caregiver review tested	E
R6	Missing, invalid and disconnected states	Validation and relay policies tested; physical outage recovery outside this campaign	S/E
R7	Roles, consent and credentials	Authentication, approved membership, TLS and revocation tested	E
R8	Modular app and accessible navigation	JVM logic, tabs and enlarged registration checked; Activity coupling remains	E/D/F
R9	Controlled development and contribution	Four main history commits establish repository-level development history	S
R10	Synthetic demonstration and improvement	Isolated Learning insights; API episodes and matched backend comparison	E/F

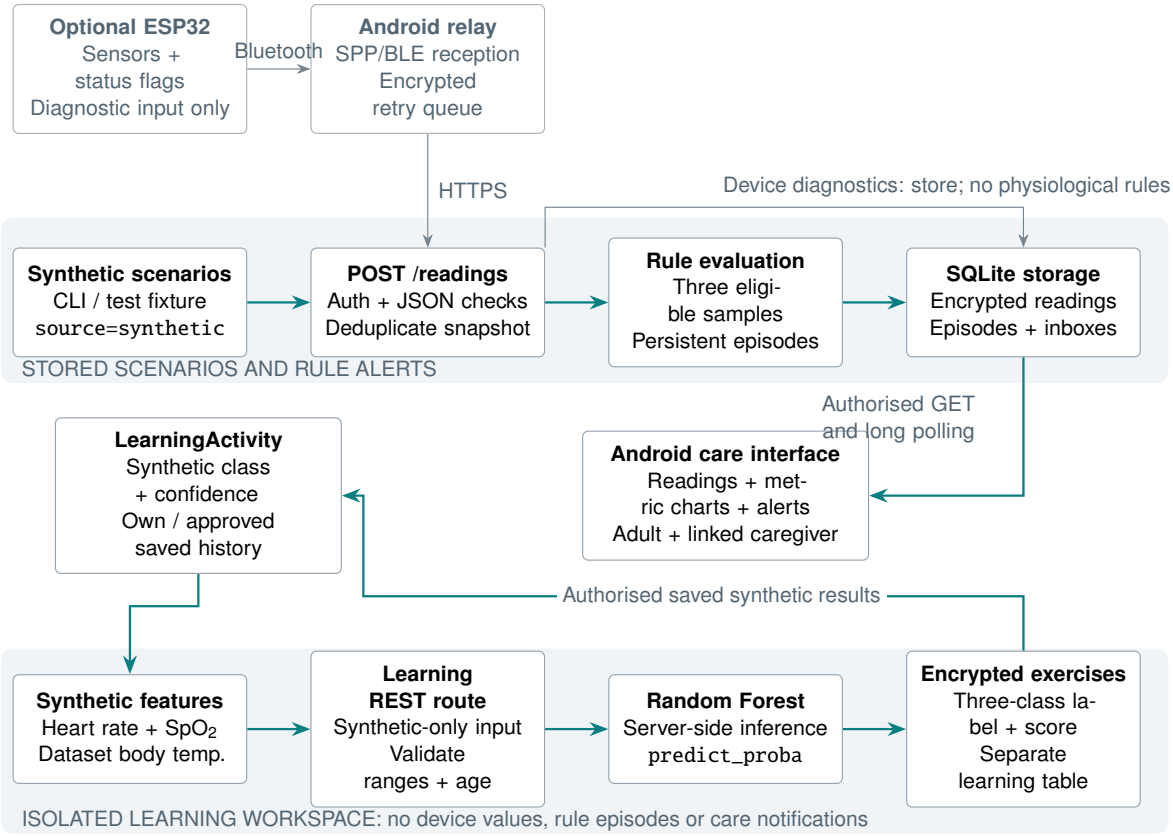


Figure 1. Current logical architecture. Teal paths describe implemented server and mobile responsibilities; grey paths describe the optional device diagnostic contract. MainActivity retrieves authorised metric history and alerts. LearningActivity presents separately stored synthetic predictions and confidence without feeding physical readings into the classifier. Test scope is recorded separately in Table 5.

3 Hardware and data boundaries

The wearable source integrates an ESP32, MAX30102 optical module, MPU6050 motion module, GY906 temperature module and photoresistor. Sensor status accompanies snapshots so that missing or warming signals need not become plausible numeric measurements. The light channel is an uncalibrated environmental indicator; darkness alone does not establish wear. Surface temperature cannot establish fever, and the installed modules do not measure blood pressure or hydration.

The optical pipeline uses a rolling sample window and validity flags; these do not establish physiological accuracy. On 27 October, an application-only upload was hash verified after a private flash backup, retaining the bootloader and partition table. The subsequent boot associated with the modem and synchronised NTP. A 65 second observation received 63 valid JSON records at 0.999 Hz, with motion and temperature modules online throughout and no resets. Fifty nine records contained clock timestamps; the largest observed clock error was 0.55 seconds. Optical readings remained unavailable without contact. Only aggregate status metadata was retained. Verified software identity and communication do not establish calibration or clinical validity.[18]

Figure 2 presents the supplied photograph of the physical ESP32 assembly, modules and wiring. It documents the assembled hardware and complements the functional architecture diagram. Visible components and connections alone do not demonstrate sensor calibration, firmware identity or successful data delivery. The historical serial and HTTP captures in Appendix C similarly establish development activity under their displayed versions. Hardware is an extension of the synthetic monitoring study, whose classification results are evaluated separately from physical measurements.

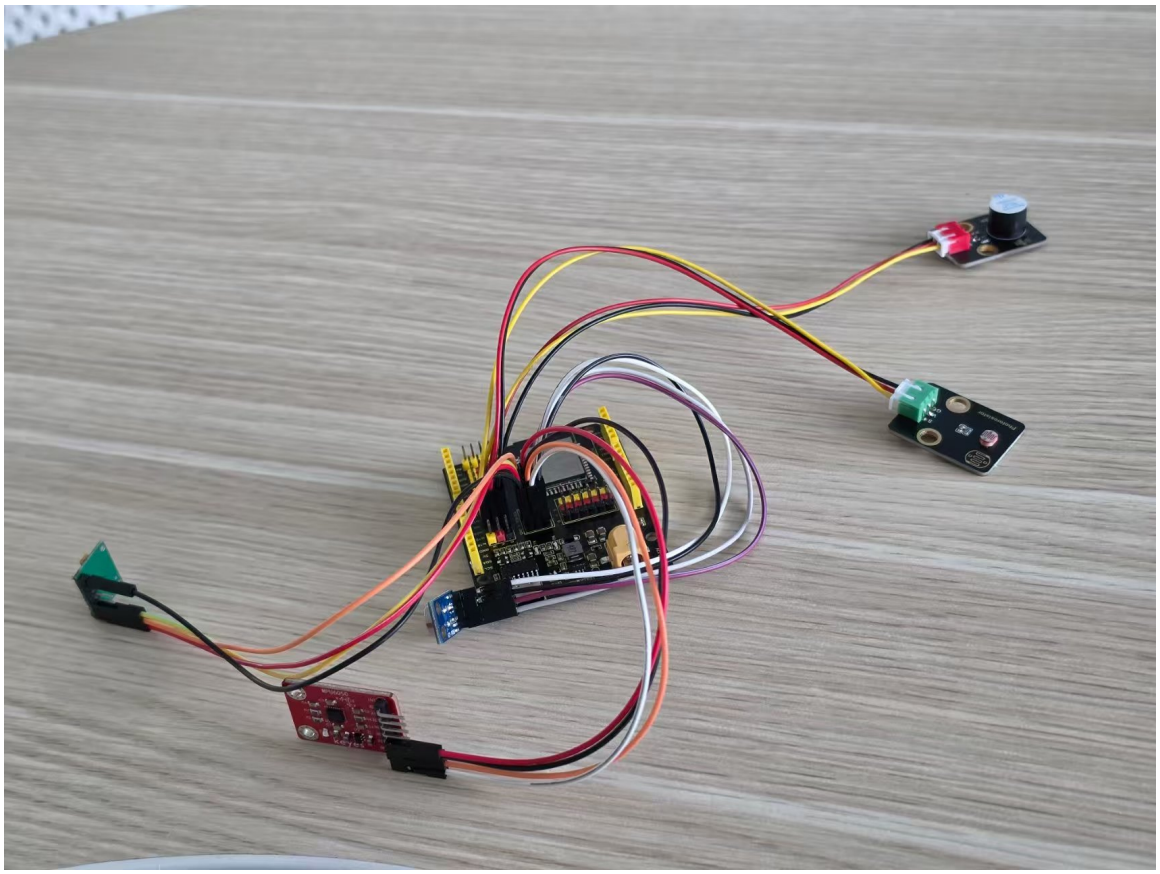


Figure 2. Physical prototype assembly supplied by the team. The ESP32 carrier is central, with jumper wiring to surrounding modules; the buzzer and light sensor boards are visible at the upper right and right. The photograph records the physical arrangement. Module runtime observations are evaluated separately.[7]

4 Mobile behaviour and programming structure

The Android application implements registration, login, adult and caregiver navigation, reading summaries, alert history and settings. Registration introduces the role before collecting name, email and password in four progressive steps. Server configuration belongs to authenticated Settings, keeping technical connection details outside the initial account form. Wearable setup is restricted to the adult role; caregivers use the relationship request and approval workflow. Separate accounts and server enforced revocation prevent shared credentials or locally hidden controls from becoming the access model.

Progressive registration presents one category of information at a time and preserves the draft when the user moves backwards. The primary action stays available while the inputs scroll, making the current step and its completion control identifiable at enlarged text. Password visibility is an explicit user choice, with masking enabled initially. These are concrete interaction decisions supported by instrumentation, rather than a claim of measured preference or reduced cognitive burden. Role selection also determines the subsequent task: the adult configures a wearable, while the caregiver seeks authorised access to another account.

The English interface combines labelled cards, text status, icons and large controls. Large metric headings, rounded grouped cards and Day, Week and Month selectors support a consistent detail hierarchy. The dated code 13 campaign on a Samsung SM-S948B with Android 17 passed 709 onboarding checks. It exercised registration at per Activity font scale 2.0 with scrollable inputs and a fixed action at least 56 dp high. These observations remain baseline evidence, rather than checks of subsequently added screens or manual TalkBack operation.[15, 5]

Figure 3 shows a layered, MVC inspired arrangement. MainActivity coordinates care screens and metric navigation; LearningActivity handles synthetic exercises. Both combine view composition with

presentation control. MetricHistoryChartView draws timestamped aggregates, while ReadingSummary, LiveSampleFreshness and AlertInbox encapsulate reusable behaviour. ApiClient and ApiSession manage requests and account state, SecureStore protects persistence, and SensorTransport separates Bluetooth implementations. This is partial separation of concerns rather than complete MVVM. Android's guidance informs the boundary between orchestration and reusable data responsibilities.[14]

Bluetooth acquisition and queued relay use dedicated services and workers. Network tasks run through an executor, with interface changes posted to the main thread. Session and lifecycle checks prevent callbacks from an earlier screen or account from becoming current state. Notification service activation uses an explicit user action and visible foreground notification. The interface instrumentation preserved permissions, preferences, session and outbox state and created no server accounts, separating controlled UI checks from a real authentication or Bluetooth campaign.

Tapping any of six metric cards opens its mounted history chart, including when the current value is unavailable. Adult and authorised caregiver views use the selected person's resource. Day, Week and Month represent rolling 24 hour, seven day and thirty day windows; earlier and later navigation changes the interval. Encrypted hourly rollups retain valid counts, sums and extrema. Weighted means preserve unequal sampling density; invalid observations remain gaps. Boundary hours are read exactly, and incomplete legacy indexing is disclosed. These properties are covered by backend tests, separately from physical sensor accuracy.

Learning insights presents editable synthetic features and Normal, Warning and Critical CSV examples. An adult can classify and save; an approved caregiver can review that person's retained exercises. Each result shows its class, model score, timestamp and source. This implements mobile presentation of confidence without silently substituting GY906 surface temperature for the model's body temperature.

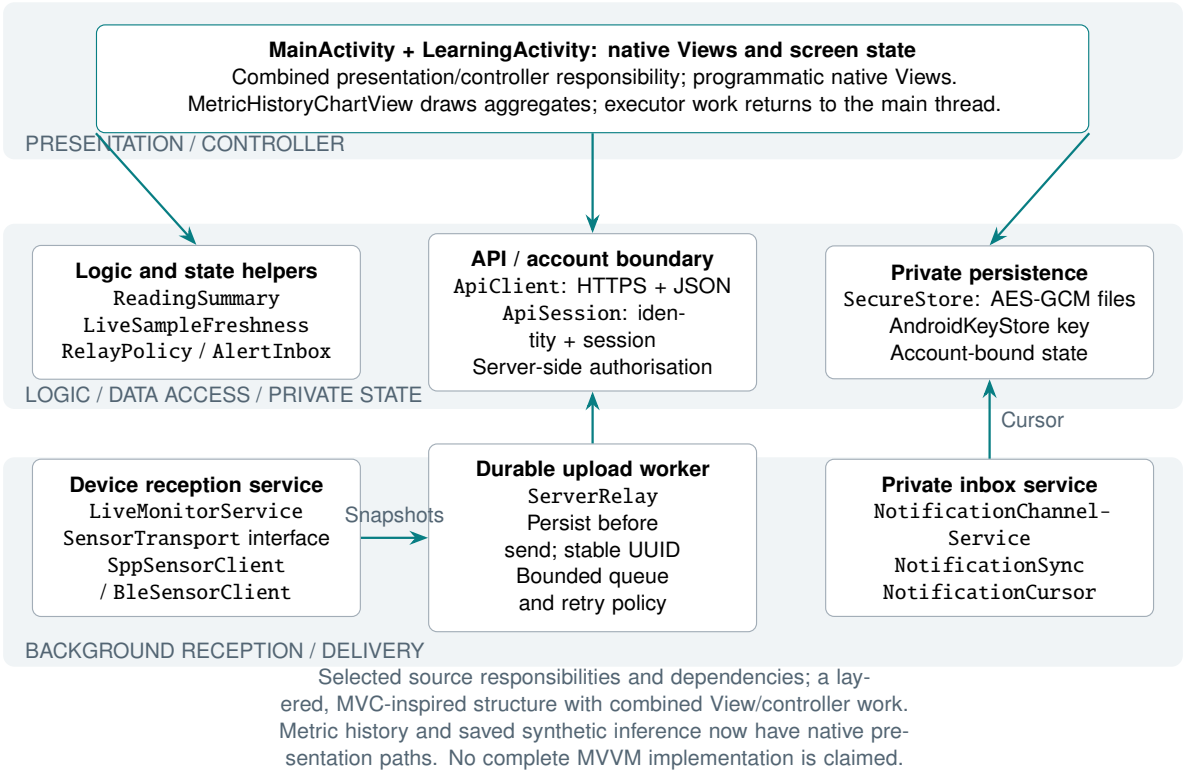


Figure 3. Actual mobile organization rather than a proposed MVVM architecture. Transport, HTTPS access, private storage and state helpers are separated from the activity, while the activity retains presentation and controller duties. Source-level separation and JVM checks do not establish Android lifecycle, accessibility, Bluetooth or background-delivery acceptance on a physical phone.

Table 2. Role behaviour in the current release. Server checks enforce access; the Android UI also restricts wearable controls to the older adult.

Action	Older adult	Approved caregiver
Read current overview and alert history	Own account	Linked adults only
Submit own sensor snapshots	Own account	Cannot impersonate adult
Approve or revoke caregiver	Own circle	Cannot self approve
Review alert	View own history	Own acknowledgement only
Account and connection settings	After sign in	After sign in
Guided wearable connection	Own wearable only	Controls unavailable
Metric chart and period navigation	Own retained readings	Linked adult's readings
Synthetic AI class and confidence	Add and review own exercises	Review linked adult's exercises
Registration accessibility	200% layout checked	200% layout checked

5 REST integration storage and alert lifecycle

The API uses the prefix `/api/v1`. Users authenticate through account endpoints and send a Bearer session token. Readings carry a stable sample identifier, capture timestamp, source and structured payload. Pydantic contracts reject malformed or unsupported inputs, while account and resource checks enforce access. Appendix A records endpoint details and an observed synthetic prediction, making the current contract explicit.[3]

The phone queue retains snapshots during temporary upload failure. Reusing a sample identifier makes a retry idempotent: a lost response creates neither another sample nor another alert. The server writes reading state, alert state and private notification entries transactionally. SQLite isolation lets readers observe committed changes; it does not guarantee delivery to a disconnected handset.[17] Figure 4 traces a qualifying upload through storage and fanout to two approved caregivers.

The outbox stores each item before transmission and removes it after acknowledgement. Its capacity is two hundred items, so the design does not imply lossless indefinite disconnection. Items retain account and endpoint ownership to prevent relay across sessions. Authentication, permission and validation errors pause sending; transient failures use retry and backoff. Policy tests exercise these decisions. Actual process termination, AndroidKeyStore persistence and subsequent physical delivery form a different device execution boundary from those tests.

Synthetic rule episodes require three recent valid samples with distinct timestamps separated by one to thirty seconds. Sustained abnormal input creates one unresolved episode per signal. Further abnormal samples update its evidence without repeatedly creating the same episode. Three eligible observations no longer triggering the rule recover it. Missing contact, disconnected sensors and invalid values cannot close an episode because unusable evidence does not establish recovery. Figure 5 makes these transitions explicit.

Acknowledgement records caregiver specific review rather than recovery. One caregiver's action does not suppress another caregiver's history. Delivery acknowledgement is also distinct from review. A persistent private inbox and authenticated long polling support recovery after interruption, while the reported timing experiments measure server processing rather than Android notification arrival.

The synchronous classification routes run through FastAPI's thread pool.[12] Bounded inference slots and per account limits constrain work. A loaded model is cached; an unavailable or busy model returns an explicit error rather than a fabricated normal result. The original classification route remains stateless. The learning route additionally stores an encrypted synthetic result with an idempotent exercise identifier and thirty day retention. Resource authorisation is checked again before committing. Model confidence does

not drive the separate rule episodes.

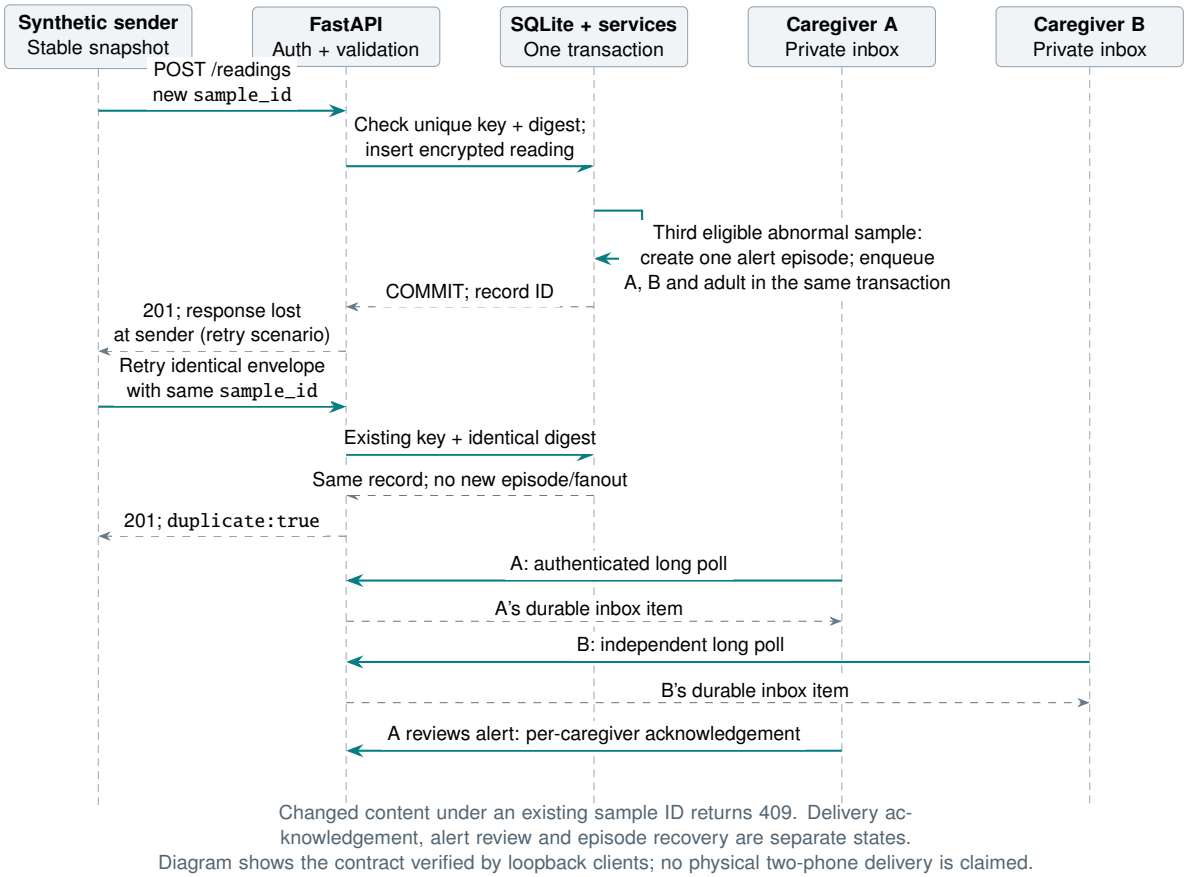


Figure 4. Reception, idempotent retry and two-recipient fanout. A new reading, any resulting rule alert and recipient inbox entries commit atomically. Replaying the same envelope returns the existing record and does not repeat alert creation. Caregivers retrieve separate inbox items; one caregiver's review does not mark another caregiver's alert as reviewed.

6 Synthetic classification and model evaluation

The experiment reads the 60,000 row repository CSV using pandas. It retains three numeric features: synthetic heart rate, oxygen saturation and body temperature. Identifiers, other measurements and alert columns are excluded from the feature matrix. Missing, nonnumeric or nonfinite values fail preparation instead of being interpreted as healthy. Targets count abnormal flags across the three alert columns: zero gives Normal, one Warning, and two or three Critical. These constructed educational categories have no independent clinical ground truth.[4]

Repeated feature tuples could place identical examples in training and evaluation. The training script groups the 18,939 unique feature tuples before a stratified split with seed 42. Every occurrence of one tuple stays in one partition. The resulting counts are 36,090 training, 12,143 validation and 11,767 test rows. This controls exact tuple leakage without creating external validation. The test partition is a local holdout, not an assessor controlled hidden test set.

Two introductory supervised classifiers are compared. Logistic regression uses StandardScaler fitted within its training pipeline, balanced class weights and at most 2,000 iterations. Random Forest uses 120 trees, maximum depth fourteen, minimum leaf size two and balanced class weights. Validation macro F1 selects the model, with Critical recall as tie breaker. Test results are reported after selection rather than used to select. Random Forest reaches validation macro F1 of 1.0000 against logistic regression's 0.7534 and is saved for endpoint inference.

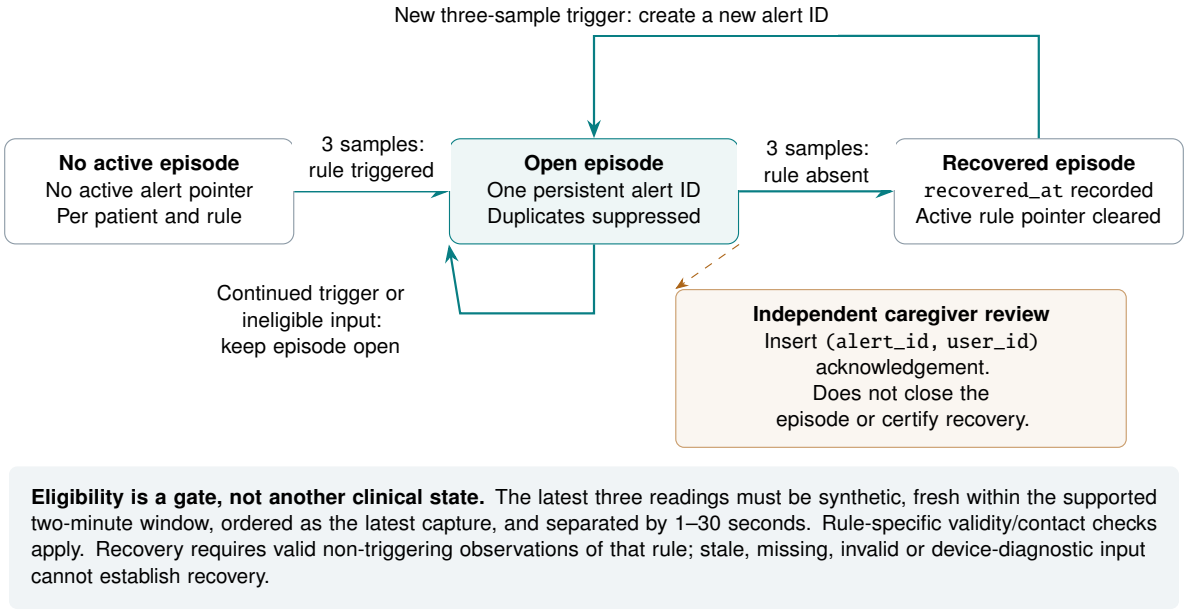


Figure 5. Implemented educational episode lifecycle for one patient–rule pair, derived from `alerts.py` and the SQLite schema. The recovered alert remains in history while its active pointer is cleared. "Recovered" describes the synthetic rule episode; it is not a health outcome. Caregiver review is an independent per-user record.

Table 3 reports accuracy and macro precision, recall and F1. Macro averaging weights categories equally; Critical recall also exposes missed critical classifications.[13] Figure 7 presents both confusion matrices with true classes as rows. Logistic regression misclassifies 905 of 3,857 Critical examples and assigns Warning to 420 of 3,366 Normal examples. Random Forest makes no observed holdout errors. Table 4 defines the corresponding rates and denominators.

The perfect forest result reflects reconstruction of deterministic synthetic boundaries rather than medical reliability. Robustness to another generator, sensor noise or distribution shift is outside this experiment. Confidence is the largest predict_proba score, rounded to six decimals. It is not calibrated against real outcomes and cannot represent the probability of disease or emergency. The inference contract requires fresh synthetic features within recorded training ranges. Before deserialisation, the local model file is hash checked and its scikit learn version compared with metadata. This mechanism does not make untrusted model files safe to load.

Logistic regression offers an interpretable baseline, but linear decision surfaces cannot readily represent every disconnected region created by combinations of threshold flags. A tree ensemble represents those combinations more directly. This interpretation is consistent with the matrices without establishing a unique explanation for each error. Reporting all classes prevents a favourable average from concealing weak Critical recall. The result is a reproducible local model comparison whose synthetic targets and dataset provenance delimit its interpretation.

Evaluation keeps feature preparation, model selection and final reporting distinct. Scaling is learned from training data rather than the complete dataset, and validation guides selection before the holdout is inspected for the reported comparison. Grouping prevents duplicate feature tuples from inflating performance through exact repetition across partitions. It does not remove the shared rule generator behind all three partitions. The resulting high score is therefore informative about reconstruction of the supplied synthetic task, while generalisation to a new population is a separate scientific question.

7 Security configuration and development evidence

Accounts use Argon2id password hashes and random session tokens with hashed server representations. Resource access requires the correct role and an active approved relationship to the adult. Invitation codes

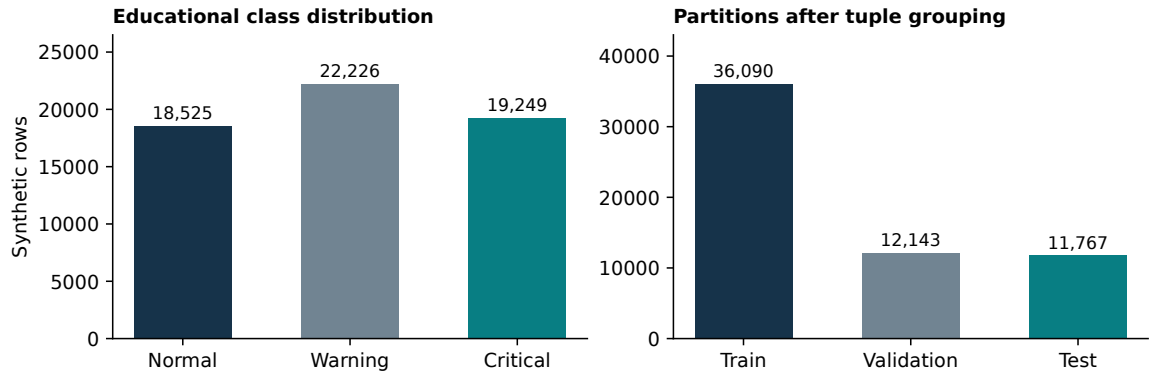


Figure 6. Class balance and row partitions from the saved training metadata. Splitting operates on unique feature tuples, so row proportions differ slightly from 60/20/20. These labels are educational categories.

Table 3. Local validation and test performance. P, R and F1 are macro averages; all values are percentages. Validation selects the model.

Model	Partition	Accuracy	P	R	F1	Critical R
Logistic regression	Validation	75.15	75.18	75.53	75.34	73.85
Random Forest	Validation	100.00	100.00	100.00	100.00	100.00
Logistic regression	Test	75.35	75.85	76.46	76.11	76.54
Random Forest	Test	100.00	100.00	100.00	100.00	100.00

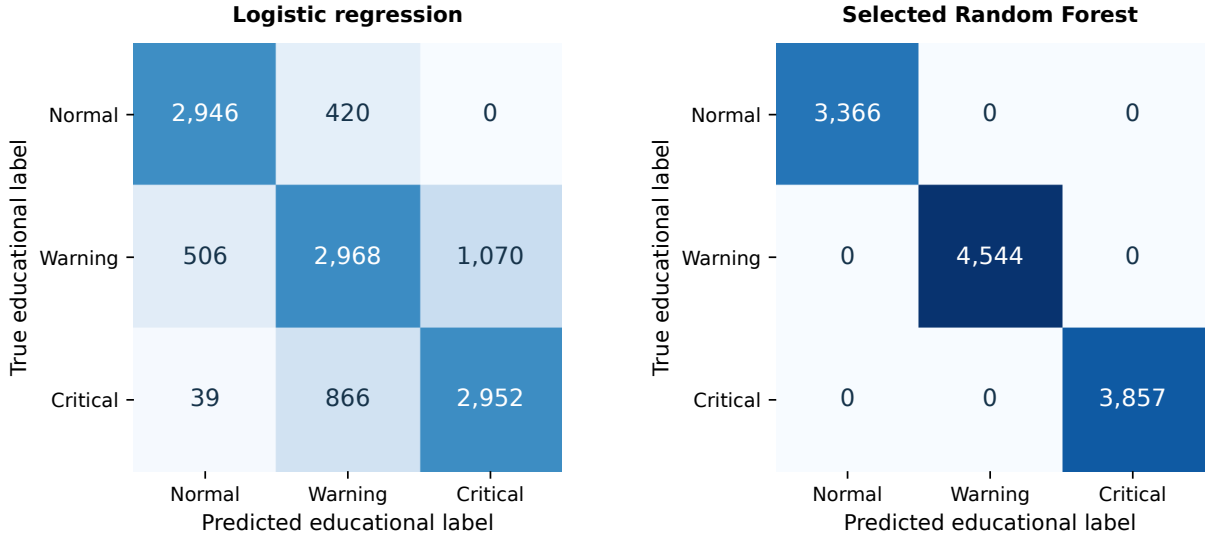


Figure 7. Confusion matrices for 11,767 locally held out synthetic rows. The selected forest reconstructs the rule derived labels perfectly on this split. This is neither clinical validation nor an assessor hidden test.

request access without independently authenticating caregivers or disclosing readings. Expiry, limited attempts and adult approval constrain guessing. Argon2id follows established storage guidance; its inclusion is an implementation property rather than evidence of a comprehensive security audit.[16]

HTTPS verifies certificate names. Sensitive server fields use authenticated AES GCM encryption, while Android protects stored state through its KeyStore backed store. Identifiers, timestamps and event types are visible database metadata. Private configuration, database state and signing material are excluded from version control. The checked in server certificate is public trust material, and backups use a passphrase with restoration invalidating sessions. These distinctions define the protection actually provided.

Table 4. Classification errors and alert observations with explicit denominators. Classifier rates do not measure phone delivery failures.

Measure and denominator	Logistic regression	Random Forest
Missed Critical classification: predicted other class / all true Critical	905/3,857 = 23.46%	0/3,857 = 0%
False Warning on Normal: predicted Warning / all true Normal	420/3,366 = 12.48%	0/3,366 = 0%
False Critical on noncritical: predicted Critical / true Normal or Warning	1,070/7,910 = 13.53%	0/7,910 = 0%
Server episode campaign	20/20 created; 0/20 duplicate episodes	
Missed Critical alert at caregiver phone	Outside server benchmark scope	
False Warning delivered to caregiver	Outside server benchmark scope	

Repository history records development under one contributor identity. It establishes versioned changes without identifying each student's personal responsibility, controlled peer review or agreed contribution factor. Appendix D presents the supplied four member roster and student IDs alongside the attribution basis. The report makes no individual authorship claim from a shared repository identity and does not infer equal effort from membership alone. This treats collaboration evidence as an attribution question separate from software correctness.[11]

8 Testing measured outcomes and improvement

Current validation passes 241/241 tests and 2,830 assertions, with 24/24 checks over loopback HTTPS. An isolated preview APK with identical production Java and resources passes 641 interface checks on Samsung S22 SM-S901E; its fictional state does not exercise real accounts or a server. Targeted suites cover history aggregation and persisted learning, including authorisation, revocation, retention and idempotency. Table 5 distinguishes these results from the dated baseline: 88 Python tests, 2,830 JVM assertions, seventeen HTTPS checks and 709 onboarding checks. Neither campaign establishes complete Bluetooth to caregiver delivery.[5, 8, 6]

The 26 October baseline benchmark uses disposable databases and an in process ASGI TestClient. Twenty five warm classification requests produce mean response time 17.34 milliseconds and p95 19.87 milliseconds. The first request takes 1,059.29 milliseconds, including lazy model loading. Twenty synthetic pulse episodes produce p95 9.09 milliseconds for third qualifying ingestion and 15.02 milliseconds from before ingestion until an authenticated inbox read completes. All twenty retry and sustained sample checks retain one unresolved episode. These timings were not remeasured for the history revision.[9]

These measurements exclude TLS, network transport, the preceding sampling interval, long polling and Android rendering. They establish local server processing rather than caregiver alert latency. Table 6 records boundaries and sample counts. Likewise, classifier false warnings and missed Critical labels describe holdout prediction errors, not complete application delivery failures. Keeping these denominators distinct makes both performance and reliability claims interpretable.

The forest improves test accuracy by 24.65 percentage points and macro F1 by 23.89 points over logistic regression. A separate chronological backend experiment compares the initial server at commit 6757c0a with the evaluated implementation. Both receive twenty distinct synthetic samples twice, representing a lost response and retry. The initial server stores forty rows; the evaluated server stores twenty, with every retry reusing its existing identifier. This removes twenty duplicate rows in the matched campaign. The adapter script documents the different contracts. The result demonstrates improved persistence semantics rather than faster operation or mobile inference integration.[10]

Scenario coverage distinguishes deterioration from malformed input. Valid synthetic sequences progress from normal values to sustained abnormal conditions and recovery. Missing fields, nonfinite values, stale

timestamps and absent optical contact instead cause rejection or preserve unknown state. Retry policies and lost responses exercise network failure decisions; unlinked and revoked caregivers exercise authorisation. Appendix B specifies the measurement protocol and separates executable software scenarios from physical transport and operating system behaviour.

The evidence supports persistent server episodes, reproducible model results and the dated account setup observations. Current source connects history charts and saved synthetic confidence to both authorised roles; its executed checks are identified separately. A successful fixture or compilation does not establish manual screen reader use, physical sensor validity or complete caregiver notification delivery. These boundaries keep implementation coverage distinct from acceptance on actual accounts and transports.

Table 5. Current revision and dated baseline verification. Counts are not added because the units and campaigns differ.

Evidence	Result	Scope and exclusions
Current Python suite	241/241 tests	Current revision software record; units are tests
Current JVM suite	2,830 assertions	Pure Java checks; not Android runtime acceptance
Metric history suite	22 tests passed	Weighted means, validity, gaps, boundaries, retention and consent
Learning record suite	14 tests passed	Synthetic persistence, idempotency, authorisation and revocation
Current Android scope	Code 16	Installed code 16 on SM-S901E / Android 13: 759 onboarding and layout checks plus direct Wi-Fi HTTPS on Network. Separately, 641 history and learning checks used the same production Java/resources in an isolated preview APK with synthetic fixtures
Current HTTPS scope	24/24 checks	Distinct from physical Bluetooth or caregiver delivery
Baseline Python/JVM	88 / 2,830	26 October records: Python tests / JVM assertions
Baseline HTTPS	17/17 checks	Loopback TLS, fictional accounts; no phone or Bluetooth
Baseline APK and UI	Code 13; 709 checks	Installed on Samsung SM-S948B, Android 17; registration at 200% font scale. APK SHA-256 `417c12ef...206378`
Baseline benchmark	20/20 episodes	Temporary databases; no duplicate retry or sustained episodes

Table 6. 26 October baseline server processing measurements in milliseconds. Percentiles use linear interpolation; no transport or Android latency is included.

Timed operation	N	Mean	p50	p95
Warm authenticated classification	25	17.34	16.85	19.87
Third qualifying reading and episode commit	20	7.08	6.95	9.09
Third reading through authenticated inbox GET	20	12.24	12.02	15.02
Cold classification including model load	1	1,059.29	—	—

The first two episode samples were already stored. Inbox timing uses the fictional adult's inbox; caregiver fanout is verified separately by the HTTPS integration fixture.

9 Conclusion

IndiCare combines local storage, explicit access control and a reproducible two model comparison with progressive Android registration. The current mobile revision adds navigable metric history and saved synthetic classifications with confidence for both authorised roles. Random Forest reproduces the synthetic holdout labels, while idempotent ingestion halves stored rows in the matched baseline retry campaign. Distinguishing model exercises, physical diagnostics and dated test campaigns makes the assessment traceable. Clinical effectiveness and complete physical caregiver delivery are not inferred from software results.

References

[1] IOT308TC. *Assessment 2 AI Enabled Mobile IoT Healthcare Monitoring Application*. Supplied assessment brief and marking rubric, `rubrik.docx`. 3,000 words ±10%; eight weighted criteria.

[2] IndiCare project. *CareMonitor 1.0.0 implementation and acceptance status*. Repository commit `5c650c1`, `README.md`, `docs/STATUS.md`, `docs/SECURITY_ETHICS.md` and `release/FIRMWARE_DIAGNOSTIC.json`. `https://github.com/deepdevjose/Older-Monitoring-Wearable`.

[3] IndiCare project. *Versioned API and implementation*. `docs/API.md`, `server/caremonitor/api.py`, `schemas.py`, `alerts.py`, `db.py`, `routes_ml.py` and Android source in `mobile/app/src/main/java/org/caremonitor/demo/`.

[4] IndiCare project. *Synthetic health model training and evaluation*. `tools/train_models.py`; `models/synthetic-health-v1/metadata.json`. Seed 42; scikit learn 1.8.0. Snapshot included as `evidence/model-metadata.json`.

[5] IndiCare project. *Release software validation, Android build and device interface checks*. `evidence/VALIDATION.json`, `ANDROID_BUILD.json`, `ANDROID_UI_EVIDENCE.json`, `pytest-current.xml` and `jvm-current.txt`, 26 October 2026. APK code 13; 709 checks on Samsung SM-S948B with Android 17.

[6] IndiCare project. *Metric history and synthetic learning revision*. `evidence/CURRENT_REVISION.tex`, `server/caremonitor/history.py`, `learning.py`, `tests/test_history.py`, `tests/test_learning.py`, `MainActivity.java`, `LearningActivity.java` and `MetricHistoryChartView.java`, 27 October 2026. Current test and device scope is distinguished from the baseline snapshots.

[7] IndiCare team. *Physical prototype assembly photograph*. Supplied 26 October 2026; reproduced as `figures/hardware_prototype.jpg`. Original source path and SHA-256 recorded in `evidence/hardware_photo.json`.

[8] IndiCare project. *Isolated HTTPS integration evidence*. `release/INTEGRATION_EVIDENCE.json`, 25 October 2026. Seventeen checks; fictional accounts; no physical phone.

[9] IndiCare report audit. *Current server processing benchmark*. `tools/benchmark_api.py` and `evidence/benchmark-current.json`, 26 October 2026. In process ASGI, temporary databases; raw samples and boundaries retained.

[10] IndiCare report audit. *Lost response retry comparison*. `tools/benchmark_idempotency.py` and `evidence/idempotency-before-after.json`, 26 October 2026. Initial `6757c0a:main.py` compared with current API using twenty matched synthetic pairs.

[11] IndiCare project. *Main branch version history*. Four commits, 25–26 October 2026; evidence in Appendix D. `https://github.com/deepdevjose/Older-Monitoring-Wearable/commits/`.

[12] FastAPI. *Concurrency and async await*. Official documentation. `https://fastapi.tiangolo.com/async/`. Accessed 26 October 2026.

[13] Scikit learn. *Metrics and scoring quantifying the quality of predictions*. Version 1.8 documentation. `https://scikit-learn.org/1.8/modules/model_evaluation.html`. Accessed 26 October 2026.

[14] Android Developers. *Guide to app architecture*. Official documentation. `https://developer.android.com/topic/architecture`. Accessed 26 October 2026.

[15] Android Developers. *Make apps more accessible Views*. Official documentation. `https://developer.android.com/guide/topics/ui/accessibility/views/apps-views`. Accessed 26 October 2026.

[16] OWASP. *Password Storage Cheat Sheet*. `https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html`. Accessed 26 October 2026.

[17] SQLite. *Isolation in SQLite*. Official documentation. `https://www.sqlite.org/isolation.html`. Accessed 26 October 2026.

[18] IndiCare project. *Controlled ESP32 application upload and Network commissioning*. `evidence/current/FIRMWARE_NETWORK_EVIDENCE.json` and `FIRMWARE_DIAGNOSTIC.json`, 27 October 2026. App-only write, private backup, verified boot, Wi-Fi/NTP and aggregate sensor status; no physiological measurement study.

A API contract and synthetic example

All paths below use `/api/v1`. A session is required except for registration and login. Exact validation and error bodies are defined by the source, not the historical screenshots.

Table 7. Selected current REST operations.

Method	Path	Purpose and access
POST	`/auth/register`, `/auth/login`	Create an educational account or start a session
GET/POST	`/me`, `/auth/logout`	Read own profile or revoke current session
POST	`/readings`	Adult uploads own source labelled snapshot; idempotent sample ID
GET	`/patients/{id}/overview`	Current reading and care state; own or approved resource
GET	`/patients/{id}/readings`	Persisted reading history; own or approved resource
GET	`/patients/{id}/metrics/{metric}/history`	Valid metric aggregates; period=day, week or month; source separated; own or approved resource
GET/POST	`/patients/{id}/learning/readings`	Saved synthetic predictions: adult adds own exercises; approved caregiver reads; thirty day retention
GET	`/patients/{id}/alerts`	Stored alert episodes; own or approved resource
POST	`/alerts/{id}/acknowledge`	Record the approved caregiver's own review
GET	`/notifications`, `/notifications/wait`	Private durable inbox or authenticated long poll
GET	`/ai/model`	Educational model identity and limitations
POST	`/ai/classify`	Synthetic features to class and confidence; no automatic reading or episode persistence

A.1 Observed inference request and selected response fields

The following normal example is retained in `evidence/benchmark-current.json`. Values are synthetic fixtures. Confidence is a model class score. The complete response also includes the model limitations.

```
POST /api/v1/ai/classify
Authorization: Bearer <session supplied at runtime>

{
  "source": "synthetic",
  "features": {
    "heart_rate_bpm": 75.0,
    "spo2_percent": 98.0,
    "body_temp_c": 36.5,
    "sample_age_seconds": 0.0
  }
}
```

```
{
  "classification": "Normal",
  "confidence": 0.999263,
  "model_id": "synthetic-health-v1",
  "source": "synthetic",
  "educational_only": true
}
```


Table 8. Expected failure behaviour of classification and data endpoints.

Condition	Behaviour to demonstrate
Missing or invalid session	Reject protected request; no patient data returned
Wrong patient relationship	Deny resource without revealing its contents
Missing feature, NaN, stale feature vector or device source	Classification rejected; no invented prediction
Same sample ID with changed content	Conflict response; existing sample preserved
Prediction limit exceeded	HTTP 429; user can retry later
Model unavailable or inference slots occupied	HTTP 503; no invented prediction is returned

B Reproduction installation and scenario evidence

B.1 Evidence package and word count

The prose count is generated by `tools/count_words.py`: 3,054 words in explicit `reportprose` environments. Headings and citation markers are excluded; including the short headings would remain within the permitted range. Tables, figures, title, references, test evidence and appendices are excluded as the assessment specifies. No source code listing appears in the main report.

Report evidence	Purpose
`model-metadata.json`	Training recipe, split, class metrics, artifact hashes and limitations
`CURRENT_REVISION.tex`	Current revision counts, APK identity and actual device scope
`benchmark-current.json`	26 October baseline timing samples and measurement boundaries
`idempotency-before-after.json`	Matched retry campaign with revision IDs and record counts
`pytest-current.xml`	26 October baseline 88 test rerun in JUnit format
`jvm-current.txt`	Baseline 2,830 assertion Java run
`ANDROID_UI_EVIDENCE.json`	Baseline 709 checks on SM-S948B; registration at text scale 2.0
`hardware_photo.json`	Original hardware photograph, source path, copy identity and scope
Report JSON snapshots	Dated code 13 build, baseline validation, integration and diagnostic records
Repository `release/` records	Current revision build, software validation and isolated interface evidence
`crop_manifest.json`	Source image hashes and exact crop coordinates

B.2 Build and isolated checks

Run the following from the repository root, with its Python environment installed. The benchmark scripts use temporary databases and fictional accounts, not the running server. They write separate rerun files rather than replacing the recorded results. The historic comparison requires Git commit `6757c0a`; it adapts the legacy unversioned API and current versioned API to the same duplicate delivery campaign. It is not a throughput comparison.

```
python3 docs/report/tools/count_words.py
bash docs/report/build.sh
.venv/bin/python -m pytest tests
.venv/bin/python mobile/scripts/test_reading_summary.py --jdk /usr
.venv/bin/python docs/report/tools/benchmark_api.py
.venv/bin/python docs/report/tools/benchmark_idempotency.py
```

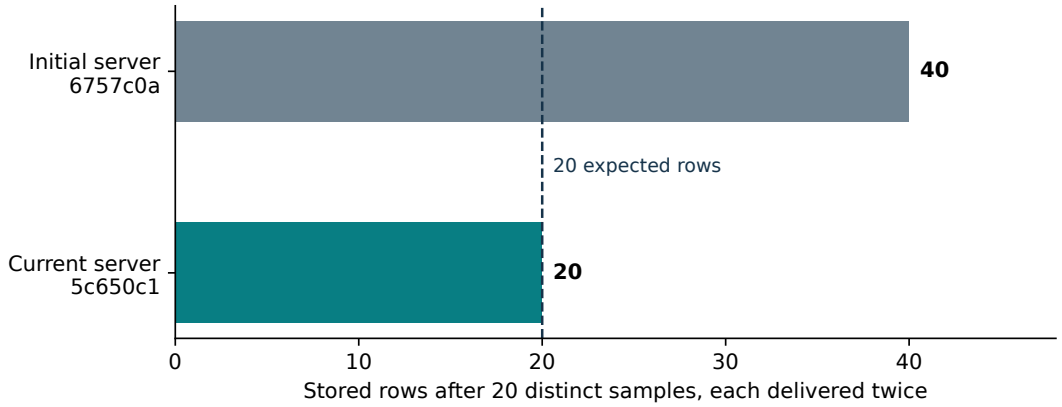


Figure 8. Measured chronological improvement in backend persistence. Forty successful responses represent twenty unique samples delivered twice. The baseline creates twenty extra rows; the current API preserves twenty unique records. The comparison excludes handset and network behaviour.

B.3 Installation and execution procedure

The current APK and source archive are distributed under `release/`. The application is version 1.0.0, code 16, minimum API 24 and target API 37. Its SHA-256 is `43646b5ff64cda4234986fd8f633f8fc7524d98a8c221d409103aa8d4f5f246c`. Device execution was recorded as follows: Installed code 16 on SM-S901E / Android 13: 759 onboarding and layout checks plus direct Wi-Fi HTTPS on Network. Separately, 641 history and learning checks used the same production Java/resources in an isolated preview APK with synthetic fixtures. The older report snapshots retain code 13's separate installation and 709-check campaign. The operational guides are `docs/SERVER.md` and `docs/PUESTA_EN_MARCHA.md`; use the exact endpoint and device identity for the selected transport.

1. Prepare the Python environment from `requirements.txt`; initialise the local server using `./carectl init -host YOUR_LAN_IP`. Keep generated keys private.
2. Use `./carectl start` and `./carectl status` to check HTTPS service readiness. Ensure the public CA bundled in the APK matches that server. Follow `docs/SERVER.md` when regenerating a build.
3. Install `release/CareMonitor-1.0.0.apk` using Android's authorised local installation flow and retain its hash and device identity. Current production checks used Android 13/API 33; the separate preview checks used Android 17/API 37. Interface tests do not grant permissions or activate sensors.
4. Use fictional adult and caregiver accounts for server experiments. Registration presents role, name, email and password as separate steps; the adult issues an invitation and approves the caregiver request. The server enforces that relationship.
5. Run `./carectl demo` with fictional adult credentials. This sends three synthetic rule samples; verify

one episode and independent caregiver review. Rule episodes are separate from saved model exercises.

6. Tap a metric card and select Day, Week or Month. Inspect the range, chart, units, gaps and coverage statement. In caregiver mode, select an approved adult and check the same authorised history. A missing current reading must not hide retained history.
7. Open Settings, then Learning insights. Under a fictional adult account choose each synthetic CSV example, inspect the three features, then classify and save. Verify class, score and timestamp in Recent exercises. The approved caregiver can read the same history but cannot submit for the adult. The original `/api/v1/ai/classify` remains a separate stateless API.
8. Reproduce interface checks only on an isolated test device or preview application, selecting the exact serial and preserving real account state. The onboarding instrumentation uses fictional in-memory profiles and has an active-session guard. Do not clear a user's app data to satisfy that guard. Current fixture evidence is scoped separately from account, backend and Bluetooth operation.

B.4 Scenario coverage matrix

Table 9. Reproducible scenarios, observed evidence and execution boundaries.

Scenario	Input and expected result	Recorded evidence and scope
Normal	Fixed synthetic normal features; return Normal and score; valid normal snapshots create no rule episode	Model/API tests and isolated LearningActivity fixture; no clinical result inferred
Deteriorating	Move from eligible normal snapshots to three distinct abnormal snapshots; one episode after persistence	Sustained condition and recovery test in isolated server fixtures
Critical	Select known Critical CSV fixture; return expected class; independently exercise an abnormal rule sequence	Held out model matrices and server episode campaign; delivery rate is a separate measure
Missing data	Omit required model feature; reject. Missing optical contact must not trigger or recover a rule	Missing feature and invalid contact tests; care views retain explicit unknown states
Invalid data	NaN, unexpected field, unsupported feature or stale age; reject classification	`test_ml.py`, `test_ai_api.py`, request limits tests
Network failure	Lose upload response and retry same ID; retain one row and active episode	Matched backend comparison and relay policies; excludes physical radio interruption
Unauthorised	Query unlinked adult, then revoke previously approved caregiver; disclose no resource	Server IDOR and revocation tests; native caregiver wearable controls are also rejected
Historical view	Mixed valid and invalid observations over Day/Week/Month; correct weighted mean and explicit gaps	`tests/test_history.py`; source separation, partial edges, backfill coverage and retention
Saved confidence	Add synthetic exercise, retry same identifier, then read from approved caregiver account	`tests/test_learning.py`; single stored result, changed-input conflict, read-only caregiver and revocation
Recovery	Three eligible observations no longer meeting the rule; recover episode; later new sustained event may create another	Sustained condition recovery test; review is not recovery

Exact automated anchors in `tests/test_api.py` include `test_sustained_condition_one_episode_valid_recovery_then_new_episode`, `test_idempotency_conflict_and_capture_order`, `test_idor_cannot_read_or_write_other_patients` and `test_revoke_removes_data_and_queued_notifications`. Model input boundaries are exercised by `tests/test_ai_api.py::test_synthetic_api_rejects_device_and_missing_features`. These names locate executable checks; they do not represent installed Android tests.

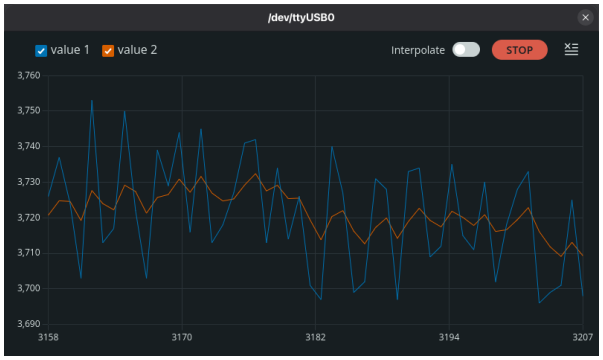
B.5 Application latency and improvement protocol

Select an initial revision and an improved revision before the experiment. Use the same device, dataset fixture IDs, server hardware, network and permissions. After identical warmup, record at least thirty trials per scenario. Measure API time from dispatch to parsed response, and caregiver latency from the first qualifying synthetic input to visible notification. Keep raw timestamps, request IDs, failed trials and timeouts. Report N, mean, p50 and p95 for both revisions; do not silently omit failed deliveries.

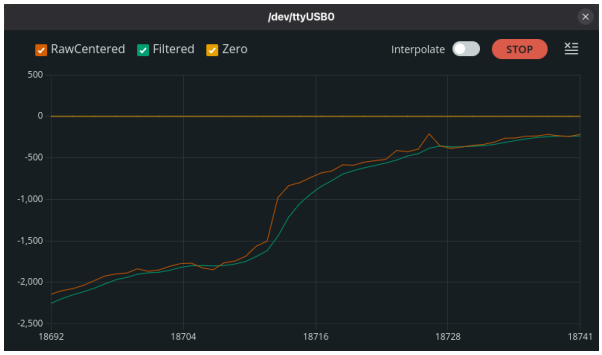
For missed Critical alerts, divide Critical scenarios without a required visible alert within the declared observation window by all Critical scenarios that should generate one. For false Warnings, divide Normal scenarios producing an unwanted Warning by all Normal scenarios. Define the observation window before testing. Model predictions and rule alerts have different semantics; a latency experiment must identify the rule output and its observation window. Separately report duplicates and data loss after reconnection.

C Historical development images

The following supplied captures are cropped for relevance. No output or values have been synthesised. Source paths and crop coordinates are recorded in `evidence/crop_manifest.json`. They document historical software experiments, separately from the assembly in Figure 2 and the Android code 13 interface campaign.



(a) Raw and smoothed ADC traces.



(b) Centred input, filtered output and zero reference.

Figure 9. Serial plotting during development on 13 October 2026. The uncalibrated ADC plots support qualitative signal conditioning observations. Axes do not establish physical time, clinical accuracy or a quantified before and after noise reduction.



Figure 10. Local API availability and successful historical uploads on 13 October 2026. The old `/sensor-data` route returns HTTP 200 while the optical module reports `no_finger`. Successful transport does not establish measurement validity, persistent storage or the final versioned API.

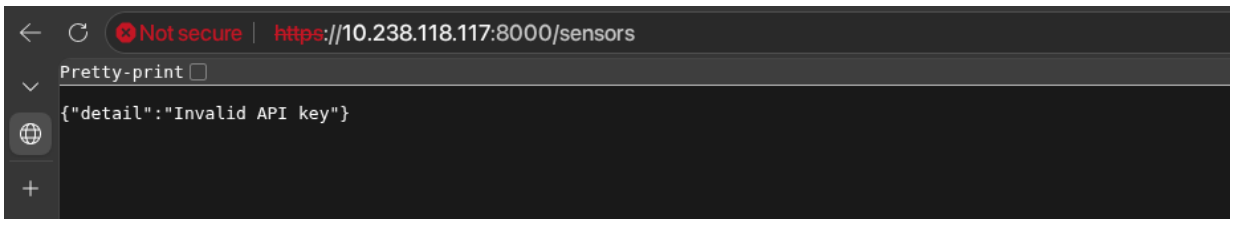


Figure 11. Historical authentication rejection on 15 October 2026. The browser reports an invalid API key and also marks the connection insecure. This establishes a negative authentication path for that version; it is not evidence of trusted HTTPS or the current account based contract.

D Project team and development history

The team roster identifies the academic submission. Repository chronology provides the technical change record; commit metadata is not used to infer a division of personal responsibilities or an individual contribution multiplier.

Team member	Student ID
Nathan Ratichon	2362789
Guannan Ge	2364229
Jose Manuel Cortes Ceron	2686162
Cheng Zhang	2361292

The tutor is Matilda Isaac. The names and IDs are reproduced as supplied by the team and are maintained in `metadata.tex`.

Date	Commit	Recorded subject
25 Oct 2026	`27a72d4`	Initial commit
25 Oct 2026	`6757c0a`	upload files
25 Oct 2026	`7e6c494`	upload csv files and data for ML part
26 Oct 2026	`5c650c1`	update: mayor fixes

The recorded Git identity is José Manuel Cortes Cerón. The four commits establish the baseline chronology. The Android interface campaign and this report revision also include local working-tree changes, identified by the release input hashes rather than represented as additional commits. This preserves the distinction between committed history, supplied documentation and the exact built artefact.

E Demonstration protocol and evaluation boundaries

The demonstration separates three observable layers: native care navigation, synthetic server evaluation and the physical bench assembly. This structure prevents a successful result in one layer from being used as evidence for a different layer.

1. Identify APK 1.0.0/code 16 and its hash. On an isolated account show four-step setup, role selection, password visibility and the fixed primary action. Explain which evidence comes from the current fixture and which comes from the historical 709-check campaign.
2. Show role-specific navigation and grouped Settings. The caregiver has no wearable connection control; the connection page is inside authenticated Settings. The adult's source-defined wearable guide separates preparation, pairing, selection and recent-sample verification.
3. Tap care metric cards and move between Day, Week and Month. Show a valid synthetic history fixture and a missing-data fixture without converting gaps to zeros. Repeat with an approved caregiver's selected patient.
4. Open Learning insights, run fixed Normal, Warning and Critical CSV examples and show saved class, confidence and source. Review the retained exercises as the approved caregiver. Explain that model scores neither classify physical sensor readings nor create care notifications.
5. Exercise a sustained rule condition with three eligible samples, inspect the stored episode and compare the two caregivers' private inboxes. The recorded HTTPS fixture establishes fanout and independent review at the server boundary.
6. Retry a sample after a lost response and compare its stored identity. Show invalid or missing inputs producing rejection or an unknown state. The backend comparison and policy tests establish these properties without assigning handset delivery latency to them.
7. Present the physical assembly in Figure 2, the source architecture and the recorded diagnostic aggregate. Explain which result comes from an image, a serial observation, a server test or a native interface check.

Table 10. Evaluation domains and the evidence supporting each claim.

Domain	Recorded evidence and interpretation	Assessment relationship
Android interface	Installed code 16 on SM-S901E / Android 13: 759 onboarding and layout checks plus direct Wi-Fi HTTPS on Network. Separately, 641 history and learning checks used the same production Java/resources in an isolated preview APK with synthetic fixtures; historical code 13 separately contributes 709 onboarding checks	Mobile functionality and programming
Classification	Two trained models, held out matrices, authenticated serving and isolated mobile display of saved synthetic scores	AI, REST and mobile integration
Persistence and alerts	241/241 tests, 24/24 checks, plus dated twenty retry pairs and twenty episode trials	Data integration and testing
Hardware assembly	Original photograph; current ten-message aggregate distinct from the older 25-message report snapshot	Architecture and demonstration
Development trace	Team roster, four baseline commits and input hashes for the local release	Configuration and documentation

MainActivity presents readings, navigable metric history, alerts and Settings. LearningActivity presents saved synthetic classifications and confidence with role enforced access. Backend tests and isolated interface fixtures establish their stated contracts and presentation boundaries; they do not establish clinical efficacy, physical sensor accuracy or complete caregiver notification delivery. A synthetic class score is not a disease probability.